

CST-407 Activity 7 Cross-Site Script Injection

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/1/2025

Table of Contents

Step 1 — Run the Victim “Comments” App (Port 8080).....	3
1.1 Import & build	3
1.2 Launch on 8080	4
1.3 Open the UI	5
Step 2 — Build & Run the Listener (“hacker”) app on 8081	6
2.1 Generate a fresh Spring Boot project	6
2.2 Run the app	7
2.3 Test the endpoints	8
2.4 Verify logs	9
Listener Console Evidence.....	9
Keystrokes File Evidence	10
Step 3 — Verify the XSS sink in the Comments app & prep the payload	11
3.1 Open the vulnerable view	11
3.2 Confirm the route that loads the page.....	12
Step 4 — Inject the Stored XSS Keylogger Payload	13
4.1 Open the Add Comment form	13
4.2 Submit the payload	14
4.3 Trigger some keystrokes.....	15
4.4 Verify in the listener console.....	16
4.5 Verify in the keystrokes log file	17
Step 5 — Apply the Fix and Verify	18
5.1 Open the vulnerable template and apply the fix	18
5.2 Restart the Comments app.....	19
5.3 Reload the Comments page	20
Step 6 — Add OWASP Java Encoder (sanitize on input)	21
6.1 Add the dependency to the Comments app.....	21
6.2 Encode user input before saving (create & edit)	22
6.3 Rebuild & restart the Comments app	23
6.4 Try to inject a <i>new</i> script (post-fix test)	24
Summary of Key Concepts.....	25
Reflection	25

Step 1 — Run the Victim “Comments” App (Port 8080)

1.1 Import & build

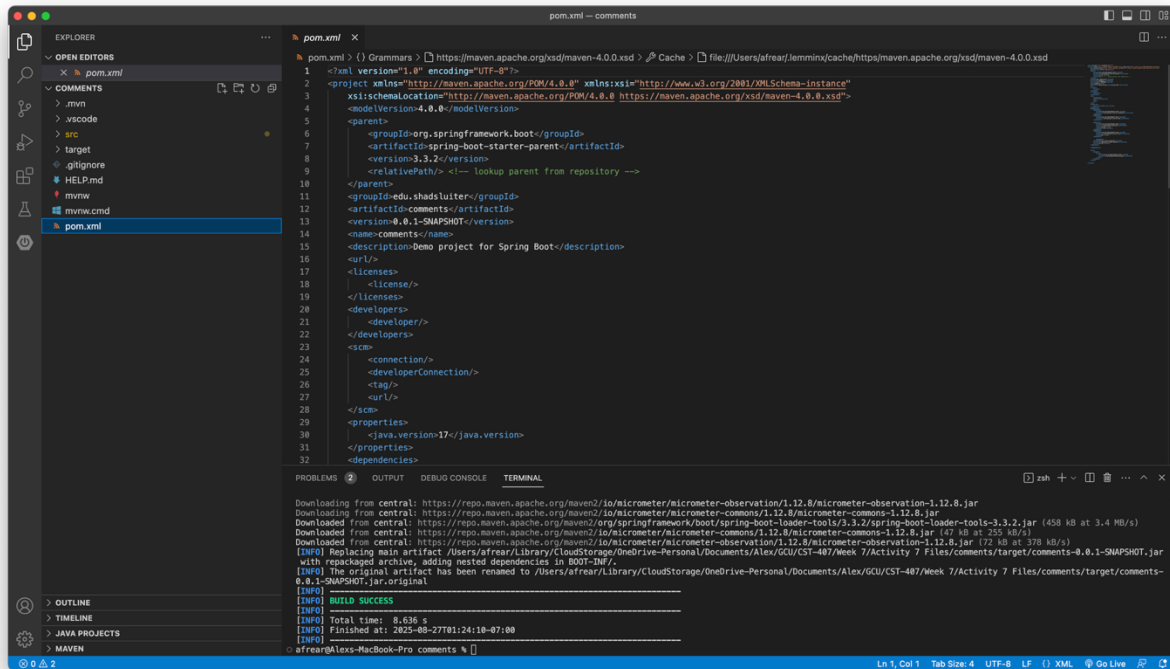


Figure 1 Project structure loaded with pom.xml, src/main/java, and src/main/resources/templates/.

1.2 Launch on 8080

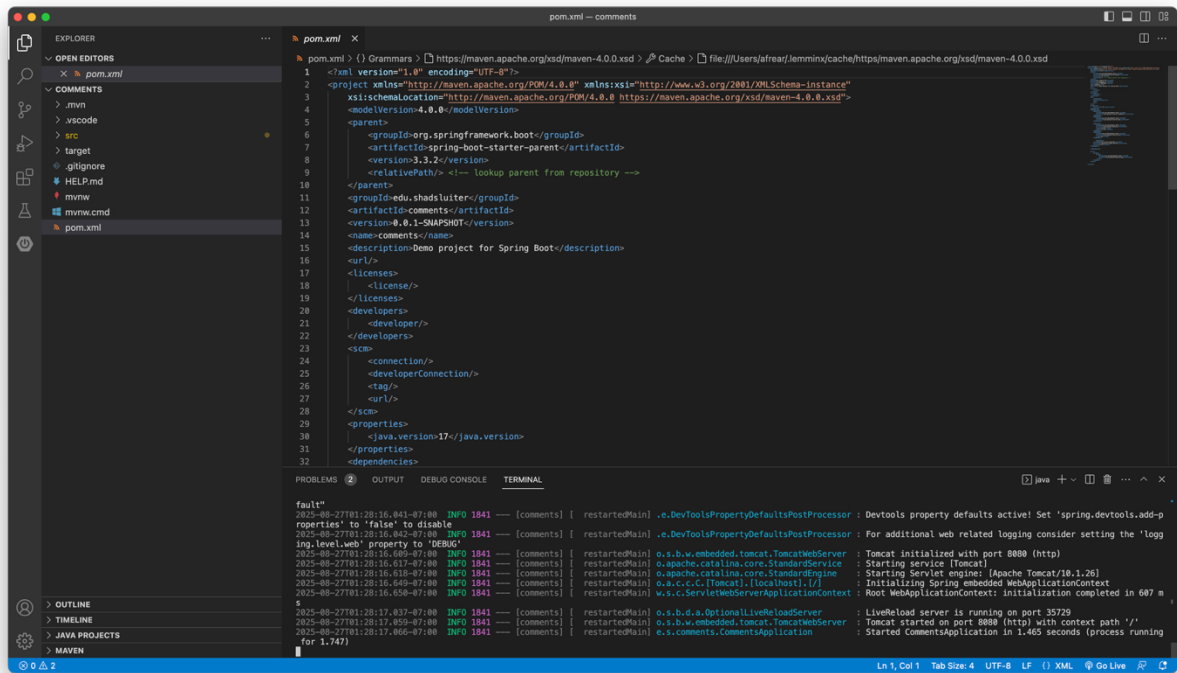


Figure 2 Spring Boot started; Tomcat listening on port 8080.

1.3 Open the UI

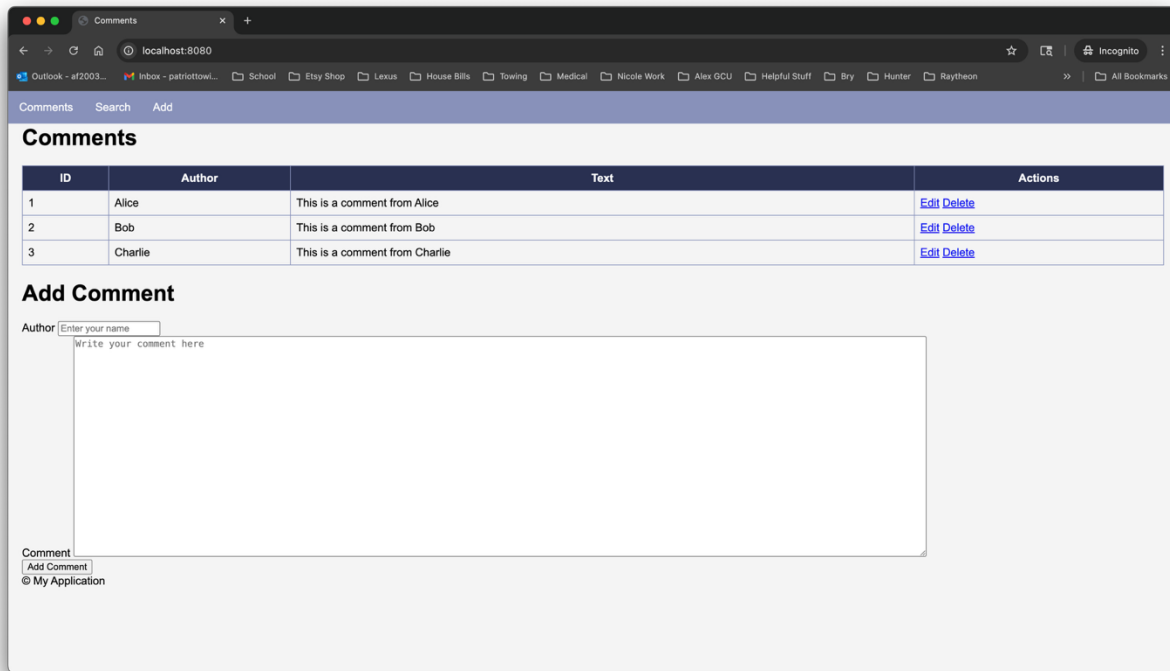


Figure 3 Comments page loaded with empty/initial table and Add Comment form.

Step 2 — Build & Run the Listener (“hacker”) app on 8081

2.1 Generate a fresh Spring Boot project

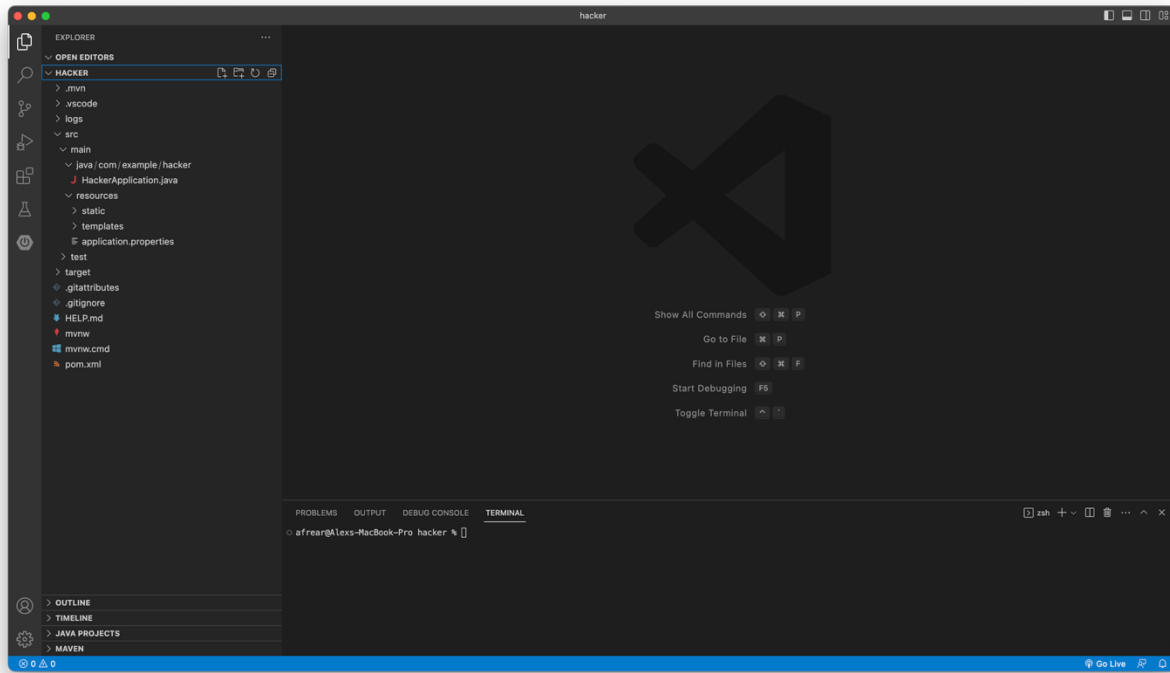


Figure 4 Project structure for the hacker listener app with controller and config files.

2.2 Run the app

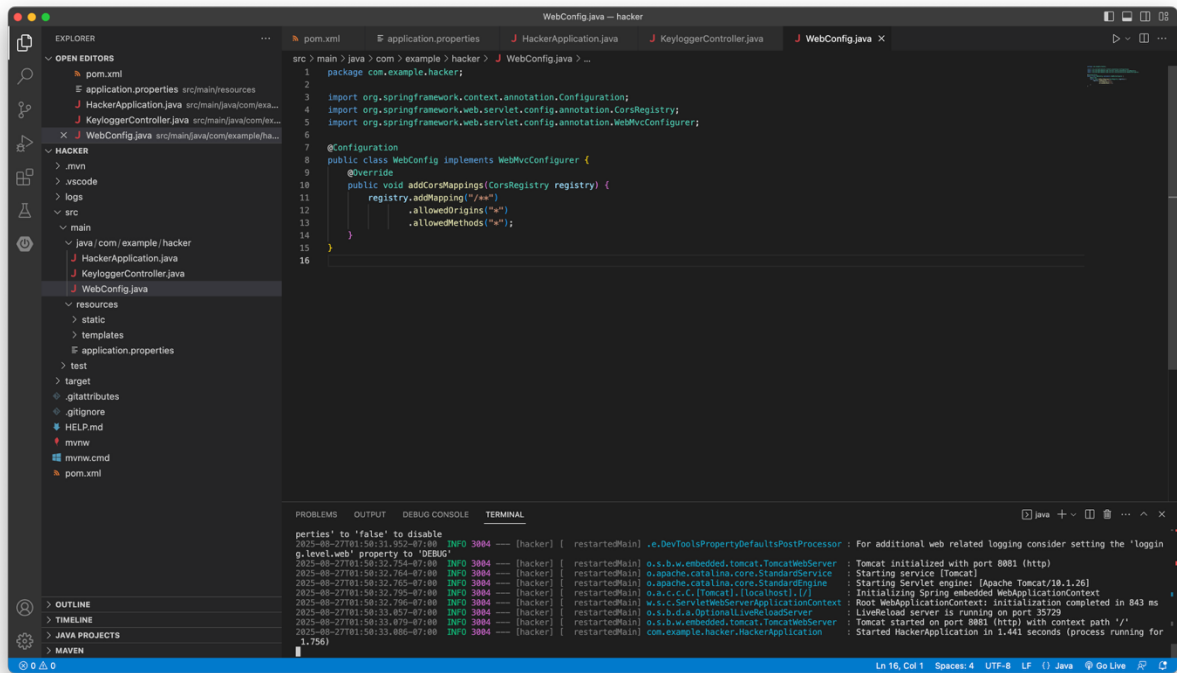


Figure 5 Spring Boot started successfully; Tomcat listening on port 8081.

2.3 Test the endpoints

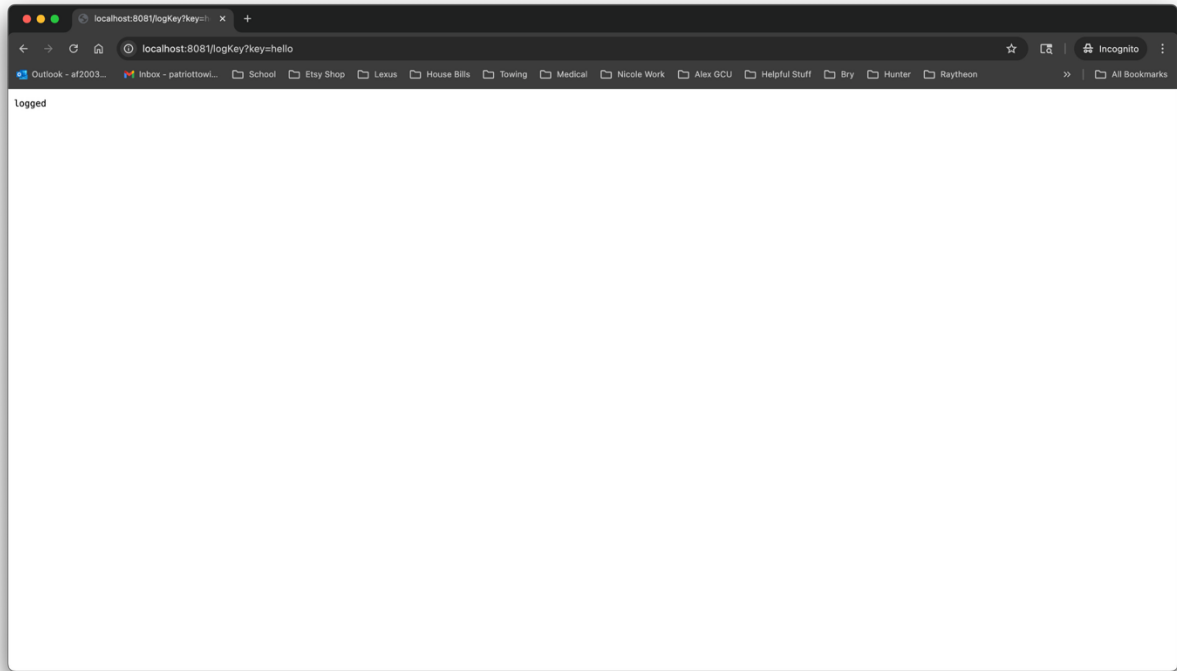


Figure 6 Browser request to `/logKey?key=hello` returns “logged.”

2.4 Verify logs

Listener Console Evidence

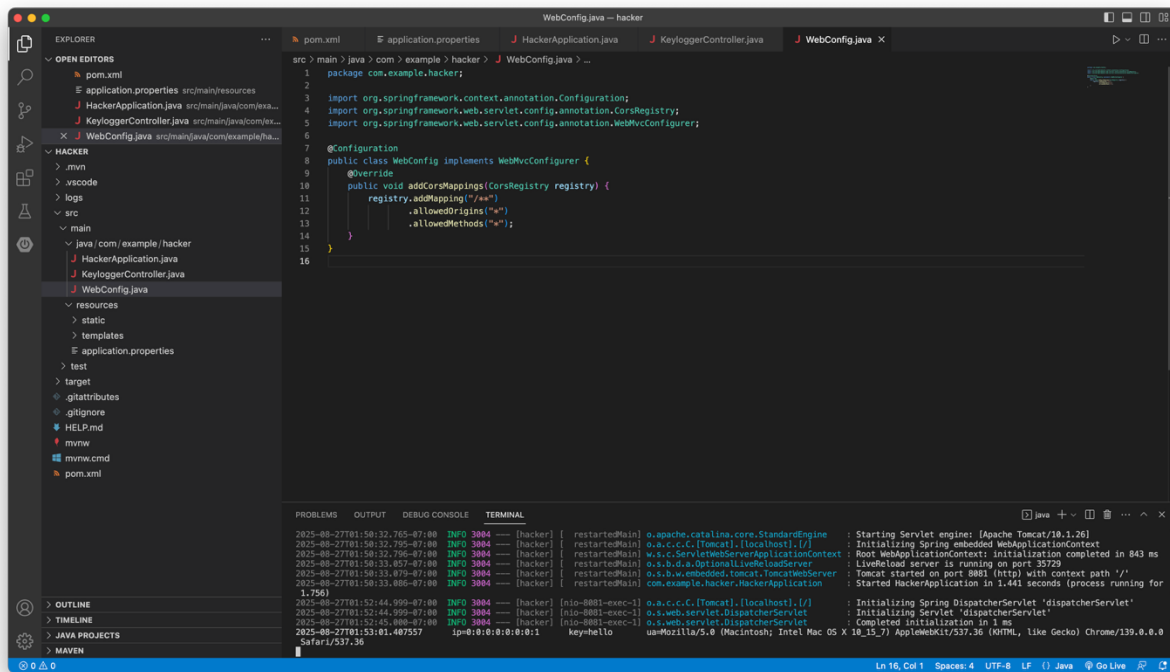
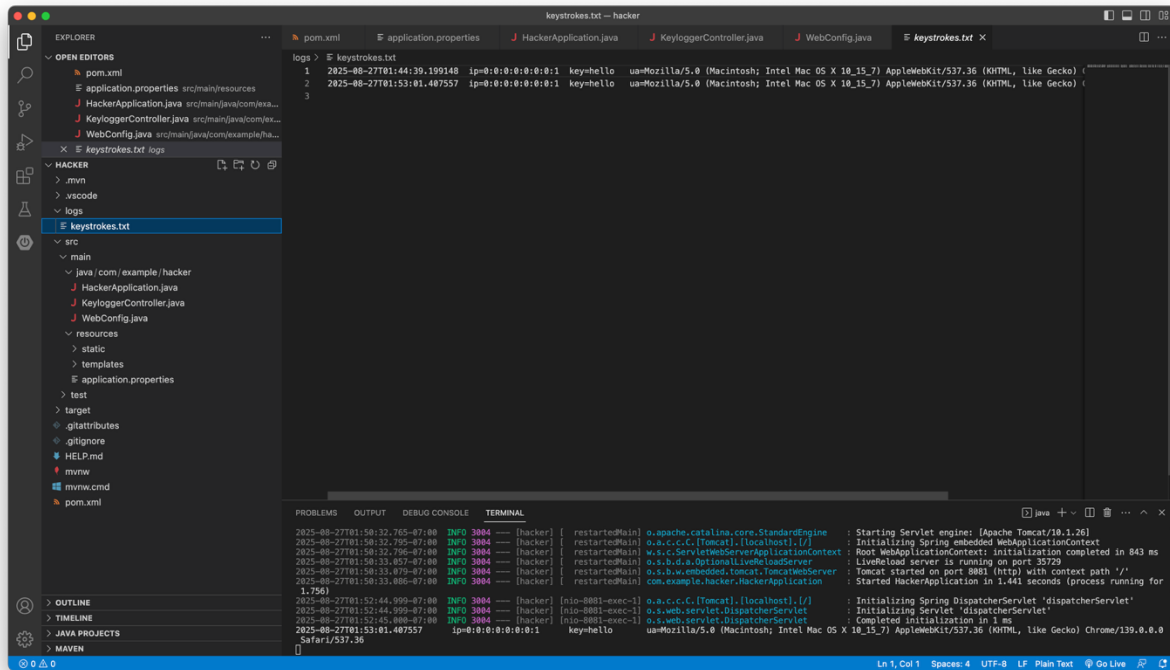


Figure 7 Console shows timestamp, client IP, and captured key value.

Keystrokes File Evidence



The screenshot shows an IDE with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The file `logs/keystrokes.txt` is selected.
- KEYSTROKES.TXT:** A text editor showing the contents of the file. It contains three log entries, each with a timestamp, IP address, and a keypress event.
- TERMINAL:** A terminal window at the bottom showing the output of the application. It includes messages about the Servlet engine, Spring initialization, and the application starting.

The `keystrokes.txt` file contains the following entries:

```
1 2025-08-27T01:44:39.199148 ip=0:0:0:0:0:0:1 key=hello ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) (
2 2025-08-27T01:53:01.407557 ip=0:0:0:0:0:0:1 key=hello ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) (
3
```

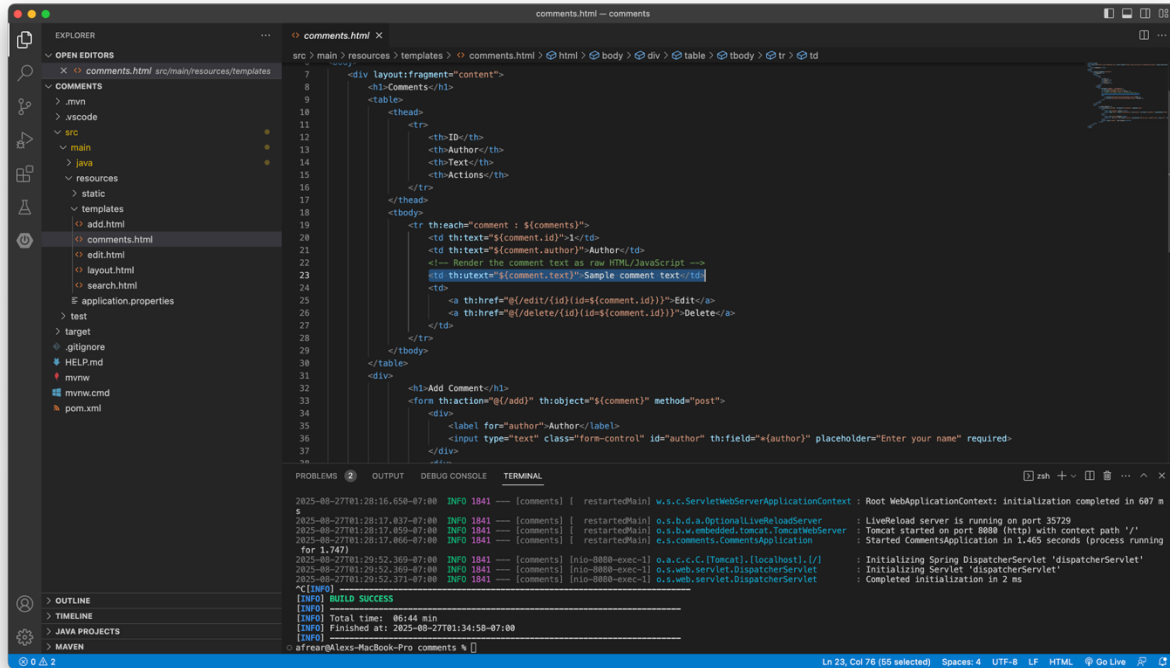
The terminal output shows the following messages:

```
2025-08-27T01:50:32.705-07:00 INFO 3804 [hacker] restartedMain o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.26]
2025-08-27T01:50:32.705-07:00 INFO 3804 [hacker] restartedMain o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-08-27T01:50:32.716-07:00 INFO 3804 [hacker] restartedMain w.a.c.s.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 843 ms
2025-08-27T01:50:33.057-07:00 INFO 3804 [hacker] restartedMain o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-27T01:50:33.070-07:00 INFO 3804 [hacker] restartedMain o.a.d.w.embedded.TomcatTomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-08-27T01:50:33.086-07:00 INFO 3804 [hacker] restartedMain com.example.hacker.HackerApplication : Started HackerApplication in 1.441 seconds (process running for 1.756)
2025-08-27T01:52:44.999-07:00 INFO 3804 [hacker] [nio-8081-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-08-27T01:52:44.999-07:00 INFO 3804 [hacker] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-08-27T01:52:45.000-07:00 INFO 3804 [hacker] [nio-8081-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
2025-08-27T01:53:01.407557 ip=0:0:0:0:0:0:1 key=hello ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
```

Figure 8 `logs/keystrokes.txt` contains appended entry for the test key.

Step 3 — Verify the XSS sink in the Comments app & prep the payload

3.1 Open the vulnerable view



The screenshot shows a VS Code editor with the file `comments.html` open. The file is located at `src > main > resources > templates > comments.html`. The code in the editor is as follows:

```
7 <div layout:fragment="content">
8 <h1-Comments-/h1>
9 <table>
10 <thead>
11 <tr>
12 <th-ID-/th>
13 <th-Author-/th>
14 <th-Text-/th>
15 <th-Actions-/th>
16 </tr>
17 </thead>
18 <tbody>
19 <tr th:each="comment : ${comments}">
20 <td th:text="${comment.id}">{comment.id}</td>
21 <td th:text="${comment.author}">{comment.author}</td>
22 <td>
23 <!-- Render the comment text as raw HTML/JavaScript -->
24 <div th:utext="${comment.text}">Sample comment text</div>
25 <td>
26 <a th:href="@{/edit/{id}{id=${comment.id}}}">Edit</a>
27 <a th:href="@{/delete/{id}{id=${comment.id}}}">Delete</a>
28 </td>
29 </tr>
30 </tbody>
31 </table>
32 </div>
33 <h1-Add Comment-/h1>
34 <form th:action="@{/add}" th:object="${comment}" method="post">
35 <div>
36 <label for="author">Author</label>
37 <input type="text" class="form-control" id="author" th:field="*{author}" placeholder="Enter your name" required>
38 </div>
39 </form>
```

The terminal output at the bottom shows the application starting successfully:

```
2025-08-27T01:28:16.650-07:00 INFO 1841 [comments] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 687 ms
2025-08-27T01:28:17.037-07:00 INFO 1841 [comments] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-27T01:28:17.059-07:00 INFO 1841 [comments] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-08-27T01:28:17.066-07:00 INFO 1841 [comments] [ restartedMain] e.s.comments.CommentsApplication : Started CommentsApplication in 1.465 seconds (process running for 1.747)
2025-08-27T01:29:52.369-07:00 INFO 1841 [nio-8080-exec-1] o.a.e.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-08-27T01:29:52.369-07:00 INFO 1841 [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-08-27T01:29:52.371-07:00 INFO 1841 [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
[INFO] BUILD SUCCESS
[INFO] Total time: 06:44 min
[INFO] Finished at: 2025-08-27T01:34:58-07:00
[INFO]
```

Figure 9 `comments.html` uses `th:utext` to render user input unescaped, enabling stored XSS.

3.2 Confirm the route that loads the page

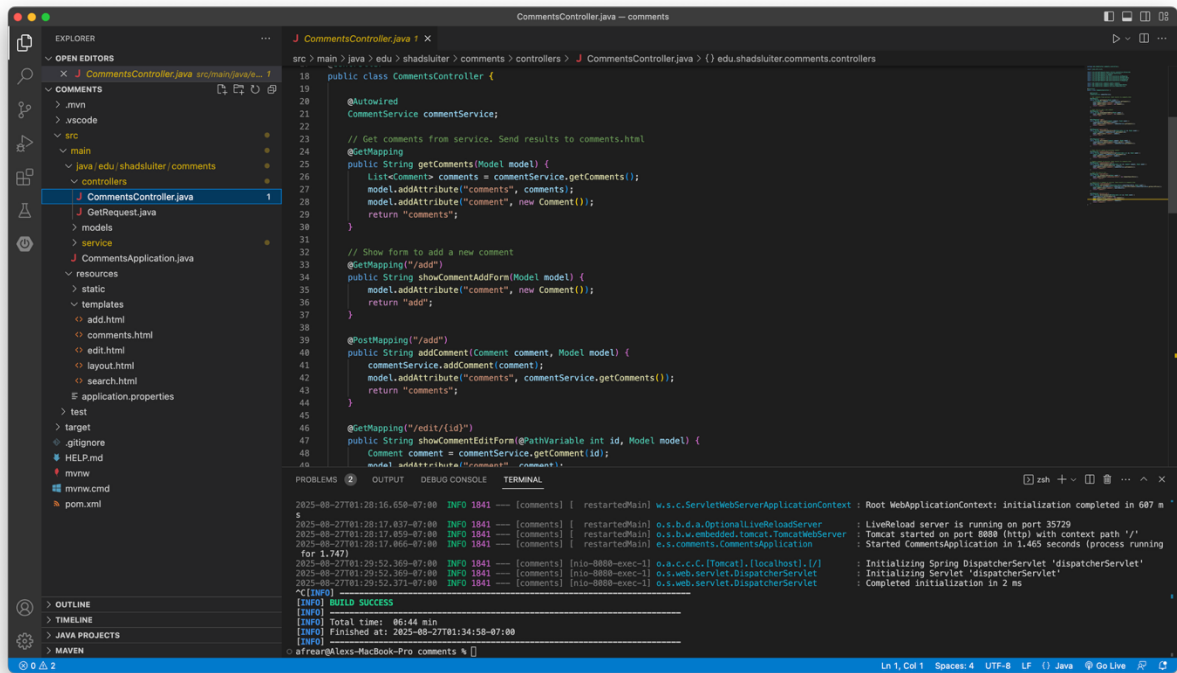
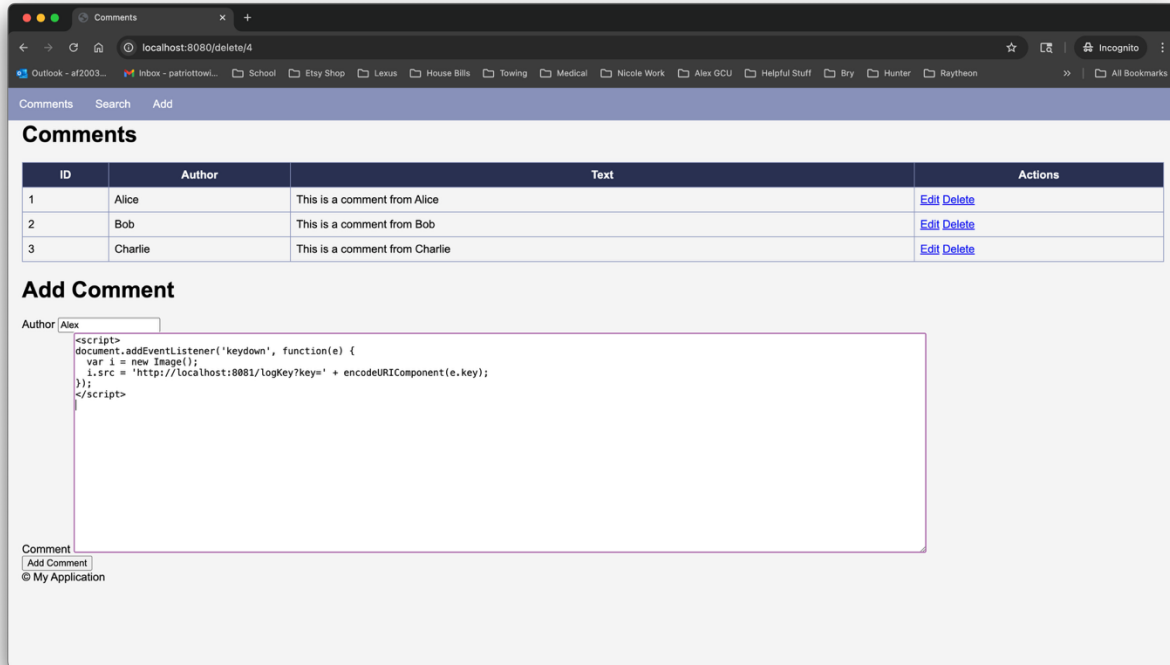


Figure 10 Controller method returns the `comments` template that displays unescaped comment text.

Step 4 — Inject the Stored XSS Keylogger Payload

4.1 Open the Add Comment form



The screenshot shows a web browser window with the address bar displaying `localhost:8080/delete/4`. The page title is "Comments". Below the title is a table with three columns: "ID", "Author", and "Text". The table contains three rows of comments. Below the table is a form titled "Add Comment". The form has two fields: "Author" and "Comment". The "Author" field is filled with "Alex". The "Comment" field is filled with a malicious XSS keylogger payload. The payload is a JavaScript code snippet that uses `document.addEventListener` to listen for keyboard events and logs the key pressed to a local file.

ID	Author	Text	Actions
1	Alice	This is a comment from Alice	Edit Delete
2	Bob	This is a comment from Bob	Edit Delete
3	Charlie	This is a comment from Charlie	Edit Delete

Add Comment

Author:

Comment:

[Add Comment](#)

© My Application

Figure 11 Add Comment form filled with the malicious keylogger payload in the Comment field.

4.2 Submit the payload

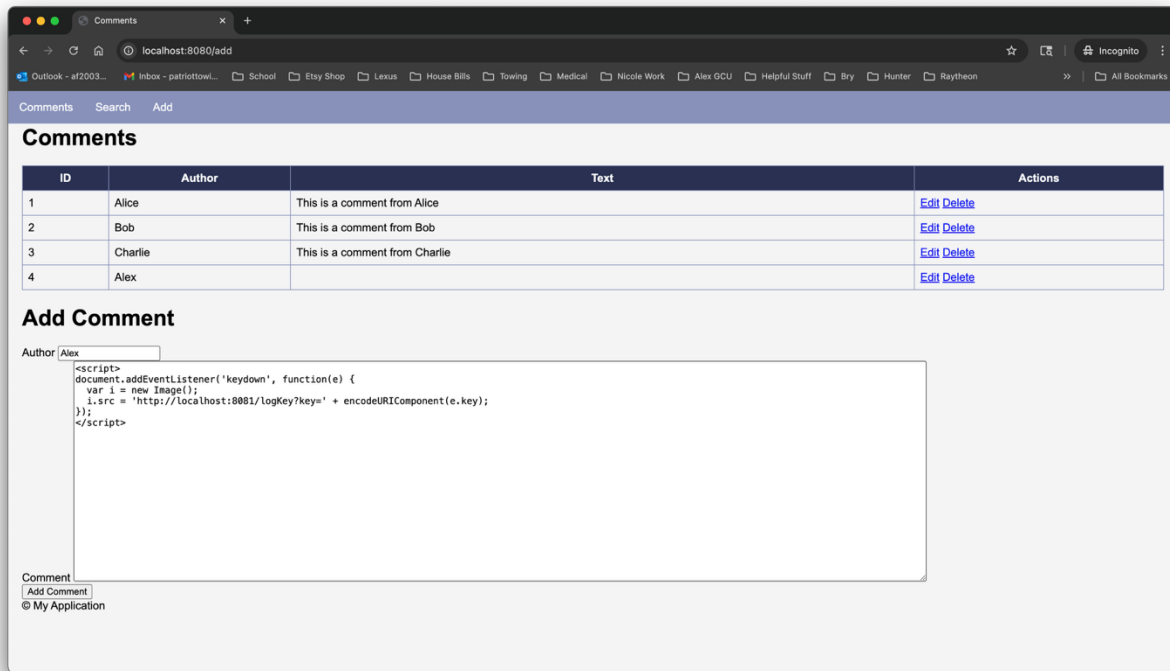


Figure 12 Comments page now contains the injected `<script>` payload displayed in the table.

4.3 Trigger some keystrokes

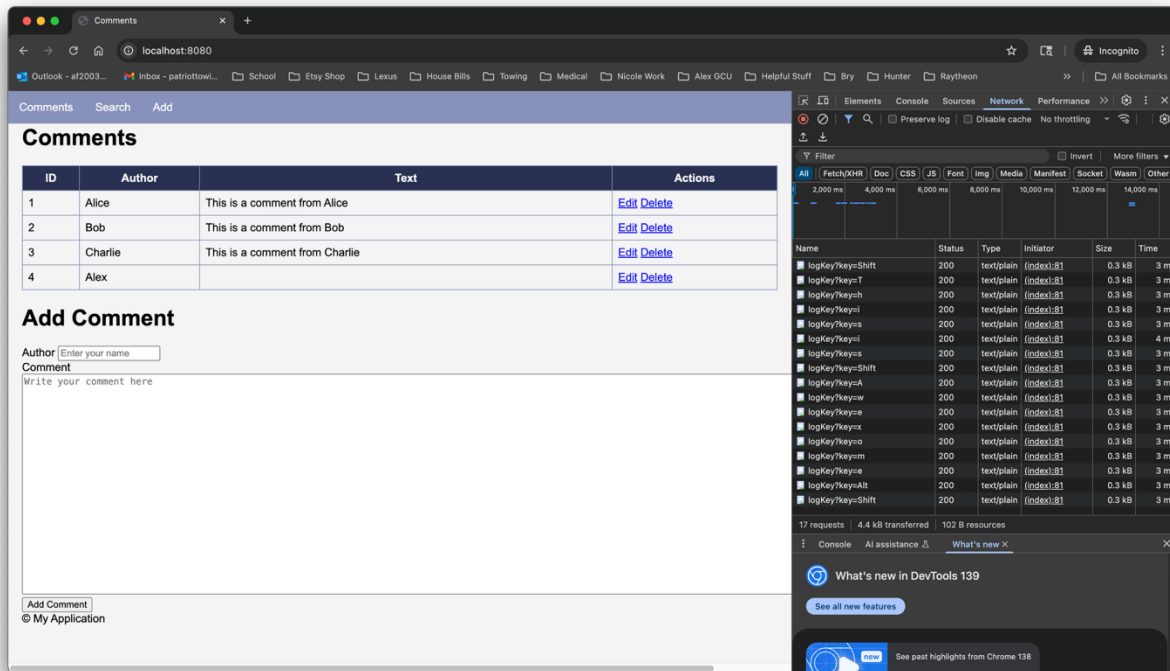


Figure 13 DevTools Network tab shows multiple requests to `/logKey?key=...` for each keystroke.

4.4 Verify in the listener console

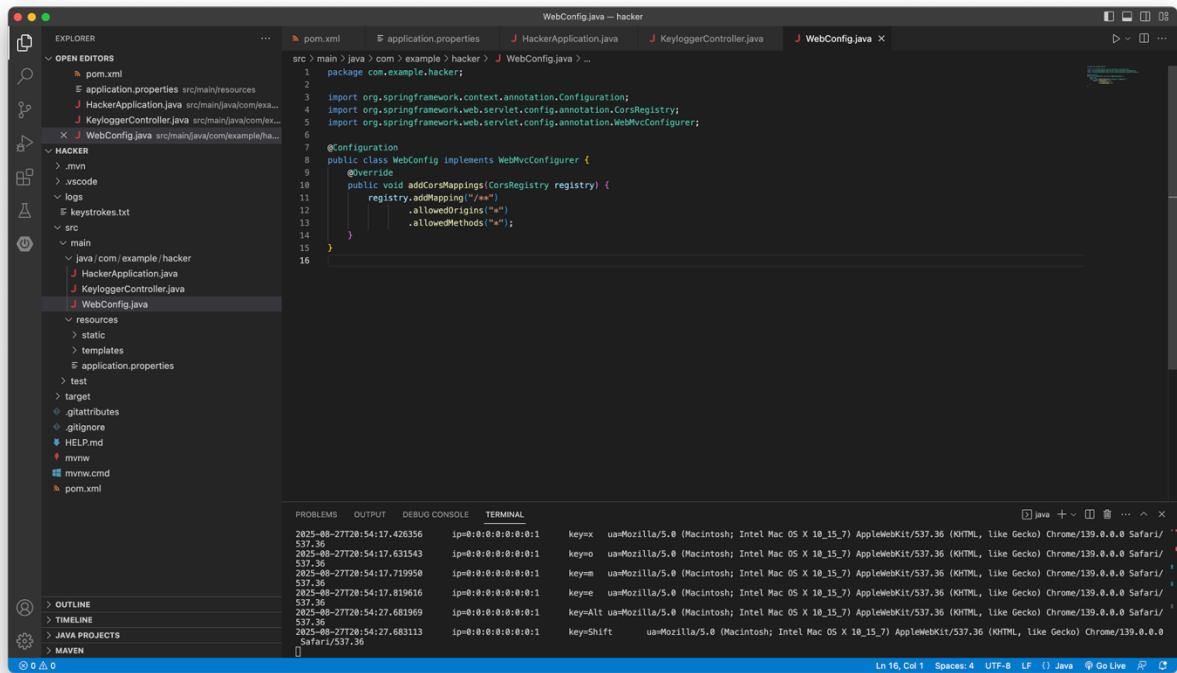
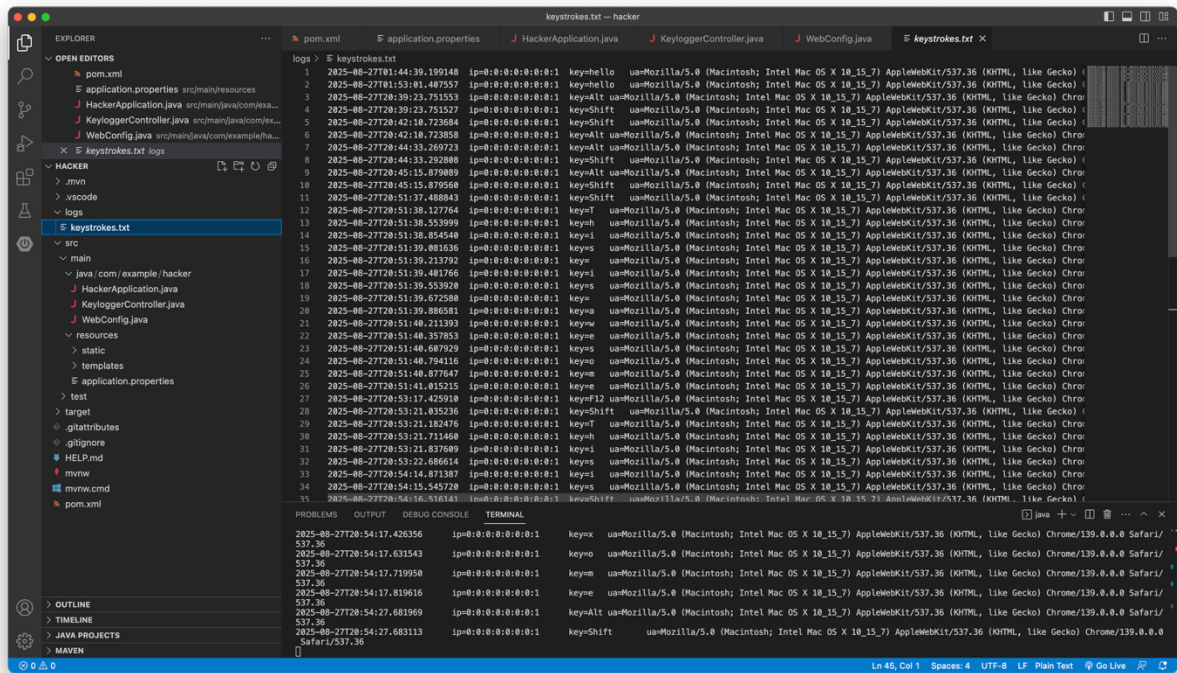


Figure 14 Console output shows each captured keystroke with timestamp, IP, and user agent.

4.5 Verify in the keystrokes log file



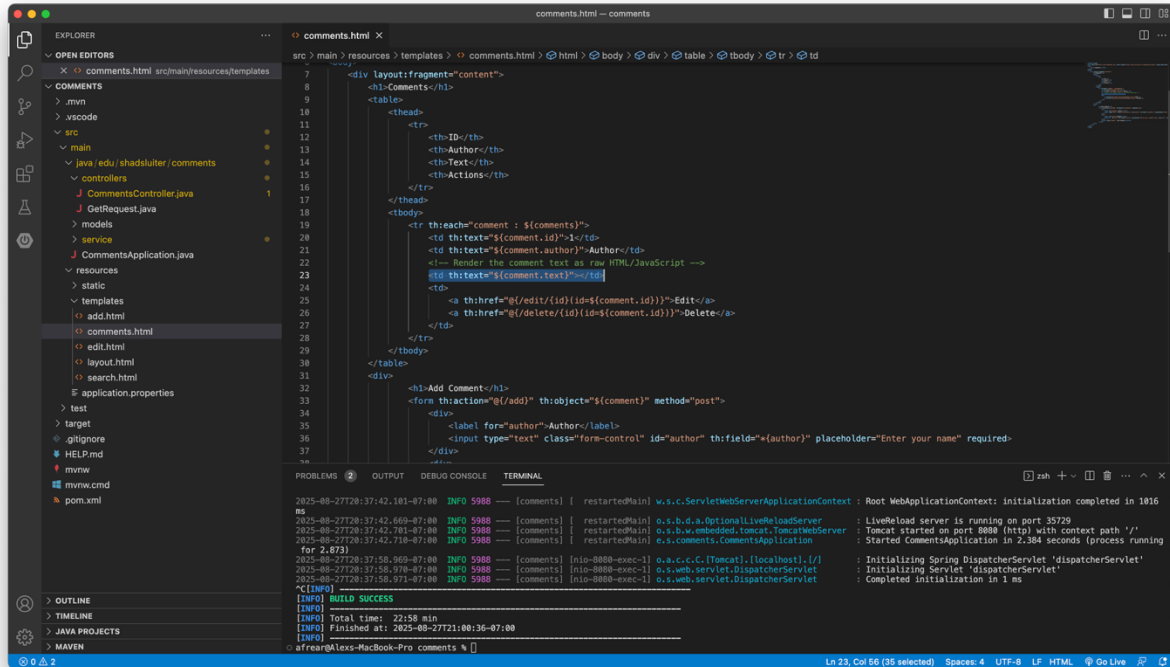
The screenshot shows an IDE with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The file `keystrokes.txt` is selected under the `logs` folder.
- LOGS:** A panel on the right displaying the contents of `keystrokes.txt`. It shows a list of log entries, each with a timestamp, a hexadecimal key code, and a description of the key pressed. For example, the first entry is: `2025-08-27T18:44:39.189148 ip=0:0:0:0:0:1 key=hello ua=Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36`.
- PROBLEMS:** A panel at the bottom showing no errors or warnings.
- OUTPUT:** A panel at the bottom showing the output of the application, which matches the log entries.

Figure 15 `logs/keystrokes.txt` contains entries for all keys typed on the Comments page.

Step 5 — Apply the Fix and Verify

5.1 Open the vulnerable template and apply the fix



The screenshot shows the Visual Studio Code editor with the file explorer on the left, the main editor in the center, and the terminal at the bottom. The file explorer shows the project structure, including the 'resources' folder and the 'templates' subfolder. The main editor displays the 'comments.html' file, which contains an HTML table with a header and a body. The header has columns for ID, Author, and Actions. The body contains a table with one row for each comment, showing the comment ID, author, and text. The text is rendered as raw HTML/JavaScript, which is the fix applied to prevent script execution. The terminal at the bottom shows the output of the application, including the startup logs and the 'BUILD SUCCESS' message.

```
src/main/resources/templates/.../comments.html <html> <body> <div> <table> <tbody> <tr> <th>ID</th> <th>Author</th> <th>Text</th> <th>Actions</th> </tr> <tr> <th>{{comment.id}}</th> <th>{{comment.author}}</th> <th>{{comment.text}}</th> <th>{{comment.actions}}</th> </tr> </tbody> </table> </div> <div> <h1>Add Comment</h1> <form> <input type="text" class="form-control" id="author" th:field="*{author}" placeholder="Enter your name" required> </div>
```

```
2025-08-27T20:37:42.101-07:00 INFO 5988 [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1016 ms
2025-08-27T20:37:42.669-07:00 INFO 5988 [main] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-08-27T20:37:42.701-07:00 INFO 5988 [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-08-27T20:37:42.710-07:00 INFO 5988 [main] e.s.c.c.c.CatWebServer : Started CommentsApplication in 2.384 seconds (process running for 2.873)
2025-08-27T20:37:50.969-07:00 INFO 5988 [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-08-27T20:37:50.970-07:00 INFO 5988 [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-08-27T20:37:50.971-07:00 INFO 5988 [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
[INFO] BUILD SUCCESS
[INFO] Total time: 22:58 min
[INFO] Finished at: 2025-08-27T21:00:36-07:00
[INFO]
```

Figure 16 Updated template escapes user input, preventing script execution.

5.2 Restart the Comments app

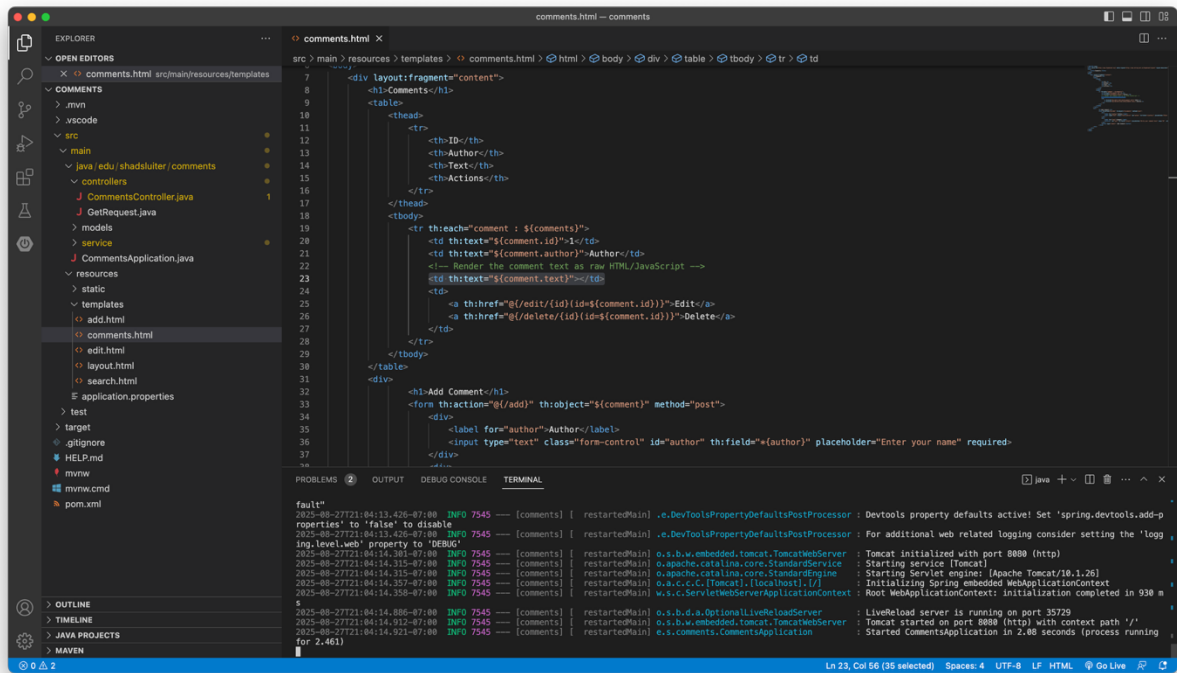


Figure 17 Console shows the Comments app restarted successfully after the fix.

5.3 Reload the Comments page

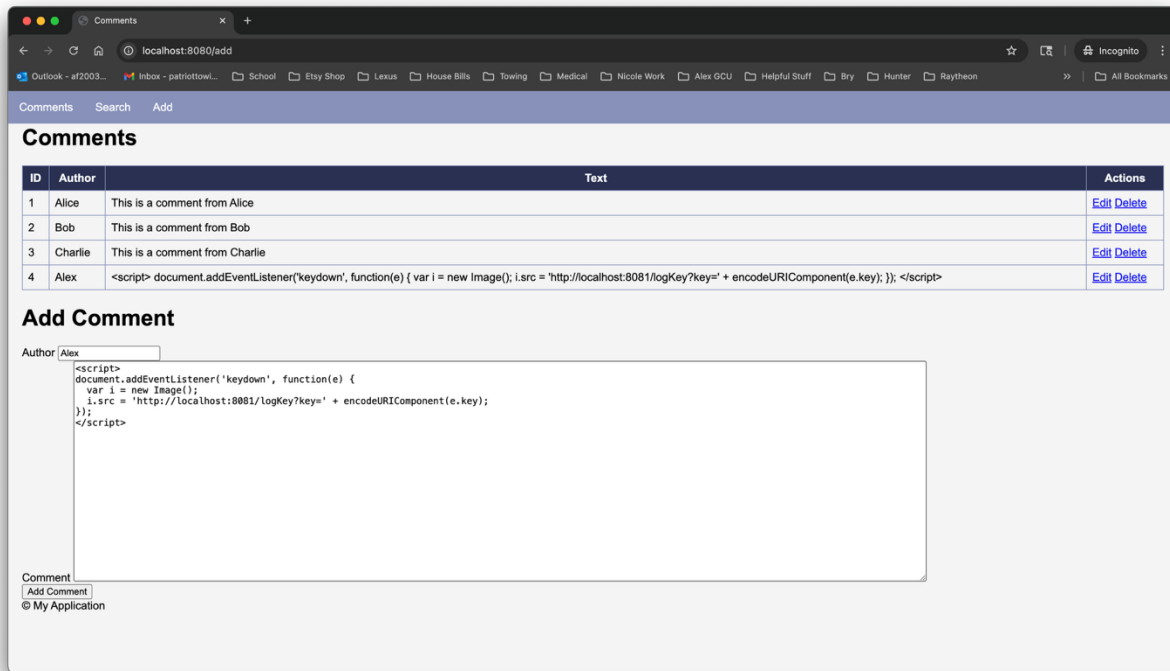


Figure 18 Comments page shows the script code as plain text instead of executing.

Step 6 — Add OWASP Java Encoder (sanitize on input)

6.1 Add the dependency to the Comments app

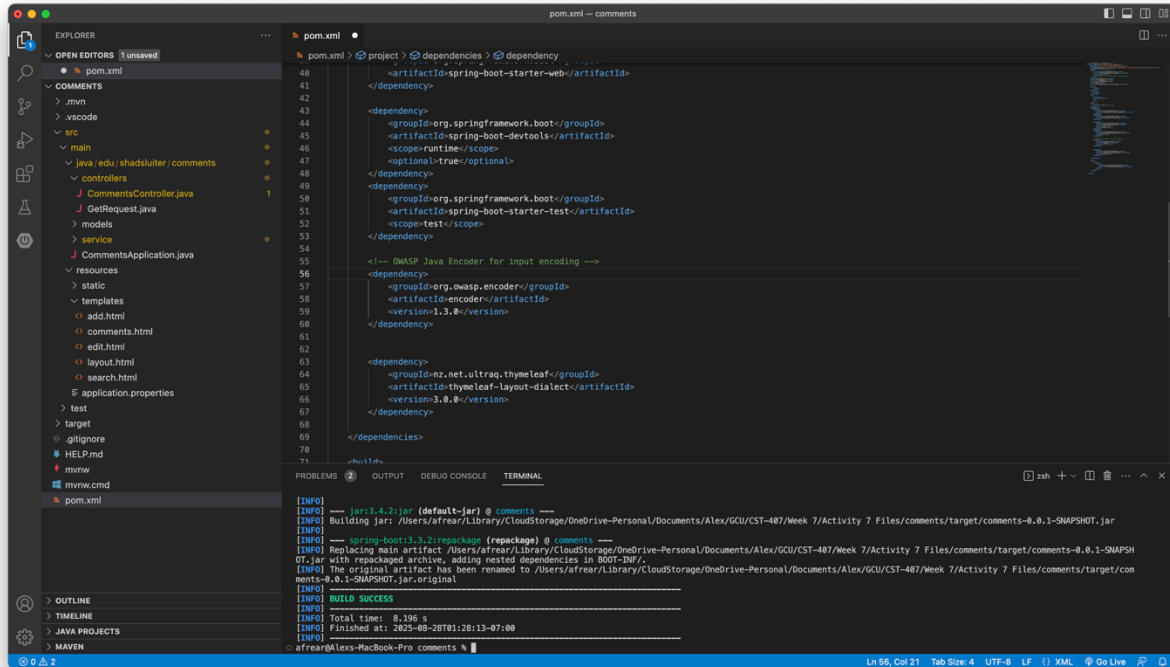


Figure 19 pom.xml includes org.owasp.encoder:encoder for input encoding.

6.2 Encode user input before saving (create & edit)

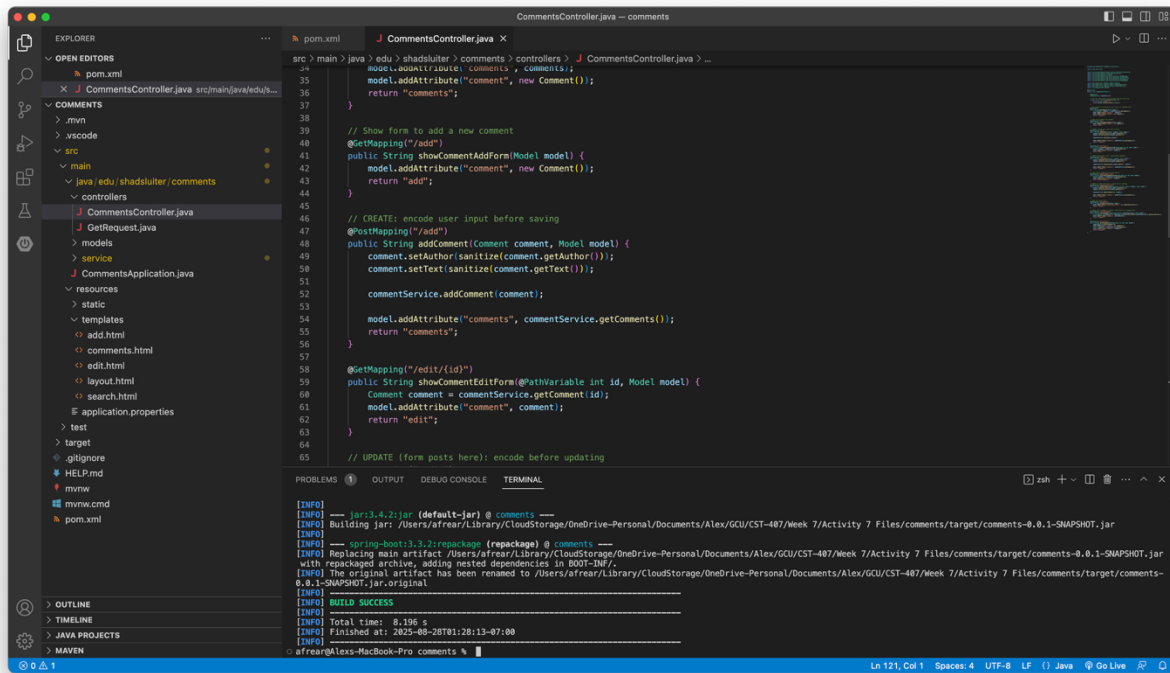


Figure 20 `EncodeForHtmlContent(...)` used to encode author and text before saving.

6.3 Rebuild & restart the Comments app

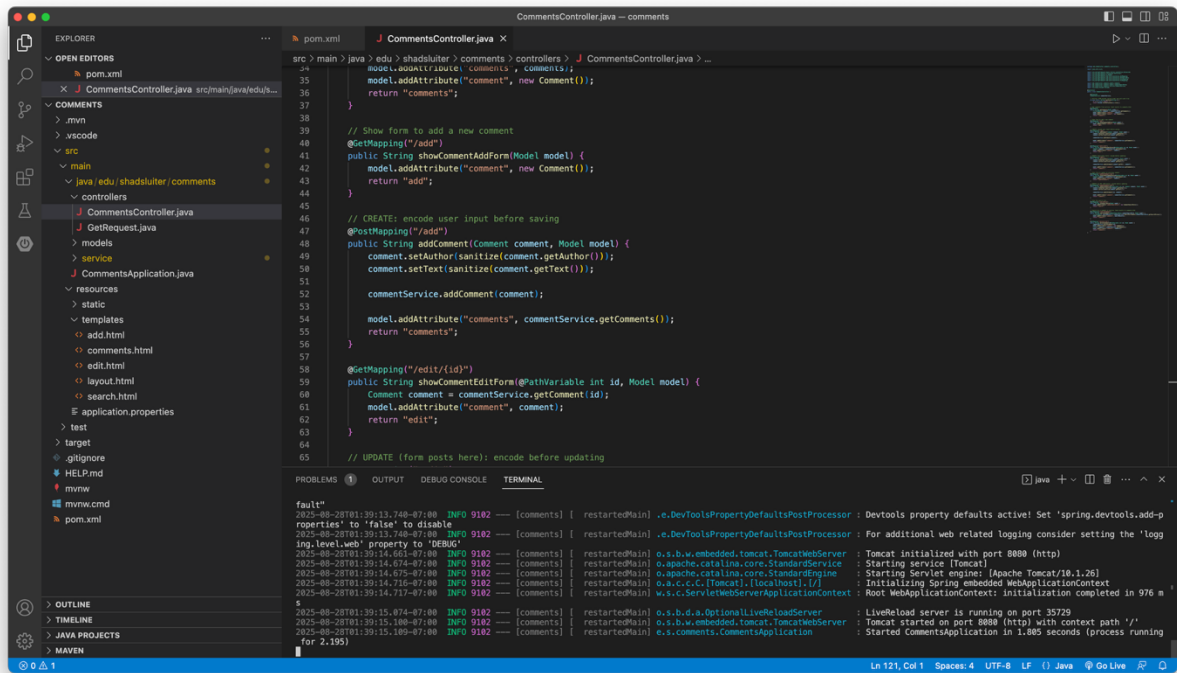


Figure 21 Application restarted after adding OWASP Encoder.

6.4 Try to inject a *new* script (post-fix test)

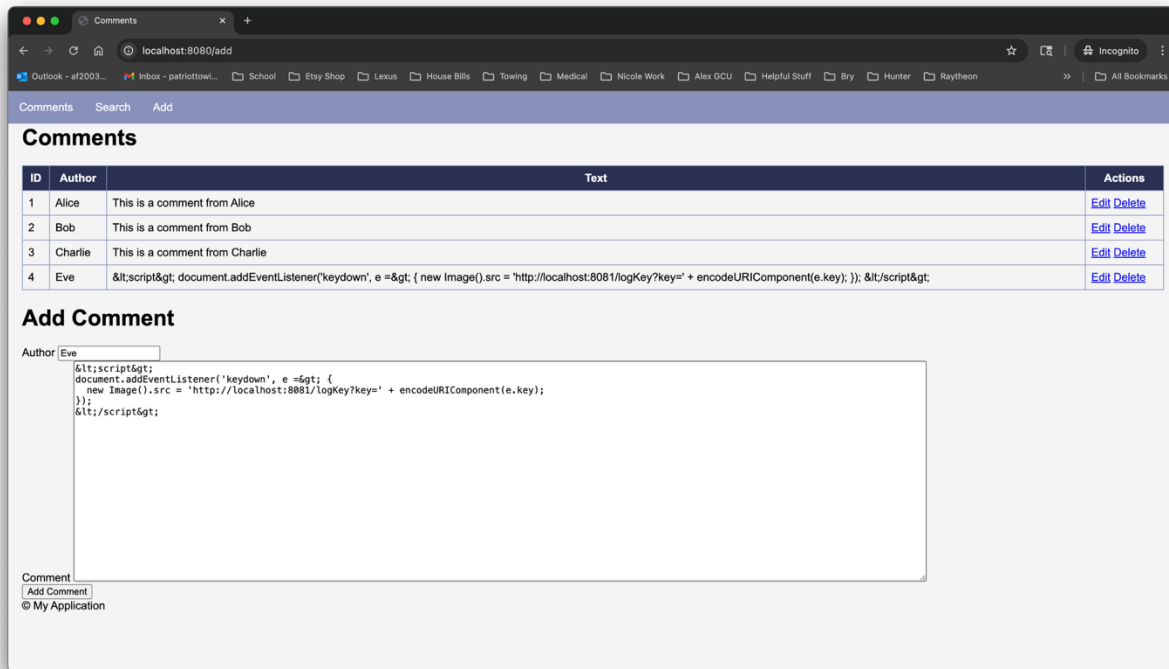


Figure 22 The newly added `<script>` comment is stored in encoded form and shown as plain text (no execution).

Summary of Key Concepts

In this activity, I demonstrated how a stored cross-site scripting (XSS) attack works by injecting a malicious `<script>` payload into the Comments application. Because the template originally rendered user input as raw HTML/JavaScript, the injected code executed in every visitor's browser, sending keystroke data to a separate "hacker" listener service on port 8081. I confirmed the attack by reviewing browser DevTools network traffic, the listener console logs, and the `keystrokes.txt` file.

To fix the vulnerability, I implemented two layers of defense. First, I updated the template to use `th:text` instead of `th:utext` so that user content would be displayed as escaped text by default. Second, I added the **OWASP Java Encoder** library to the project and updated the `CommentsController` to sanitize user inputs with `Encode.forHtml()` before storing them. With these changes, malicious scripts were neutralized and displayed safely as plain text, proving the application was no longer vulnerable.

Reflection

This lab reinforced how dangerous stored XSS can be, since the injected script executed automatically for every visitor who loaded the Comments page. It also showed how attackers can easily capture sensitive data like keystrokes with a separate listener service. By setting up this attack end-to-end, I saw how real the risk is when user input is not handled securely.

Applying the fixes emphasized the importance of **defense in depth**. Using `th:text` ensures safe output encoding at the view layer, while OWASP Encoder enforces sanitization before data is stored, reducing the risk of script injection from multiple angles. This dual approach highlights why developers should never trust user input and should always validate, sanitize, and escape data. The activity ultimately deepened my understanding of both exploiting and mitigating XSS vulnerabilities in web applications.